

Week 3 Introduction

Mathematics for Multidimensional Data

✓ **Video:** 3.1 Multidimensional Data Points and Features
2 min

✓ **Reading:** Multidimensional Data Points and Features Recap
10 min

✓ **Practice Quiz:** Multidimensional Data Points and Features – Review Information
1 question

▶ **Video:** 3.2 Multidimensional Mean
2 min

✓ **Reading:** Multidimensional Mean Recap
10 min

✓ **Practice Quiz:** Multidimensional Mean – Review Information
1 question

▶ **Video:** 3.3 Dispersion: Multidimensional Variables
3 min

✓ **Reading:** Multidimensional Variables Recap
10 min

✓ **Practice Quiz:** Dispersion: Multidimensional Variables – Review Information
3 questions

✓ **Video:** 3.4 Distance Metrics
5 min

✓ **Reading:** Distance Metrics Recap
10 min

✓ **Practice Quiz:** Distance Metrics – Review Information
4 questions



Normalisation Recap

Normalisation, also known as **standardisation**, is a way of applying the same scale to all features. If you have measured two features for each data point, but one varies between 0 and 1000 and the other between 0 and 1, you start adding these two together and making calculations, you can see that the first feature dominates the second ones. This is very obvious when we think about proportions: 1 for the first feature, and 1 for the second one. If suddenly the first one were to be halved, it would still be 1. Conversely, if the second variable is halved, it is only going to be 0.5 (e.g. 900). The second one is so small that its changes won't really be visible when you add them together. That's why, that, you transform them so that they are measured *on the same scale*, between 0 and 1, shrinking or increasing your variables, but respecting all proportions (e.g. 0.5 and the second 0.5). In this case, halving one variable (to 0.45) or the other would have the same result!

The method is the following: you **subtract the mean**, and **divide by the standard deviation**.

$$X_{ij}^Z = \frac{x_{ij} - \bar{x}_j}{s_j}.$$

This will transform your data so that it now has a **mean of 0** and a **variance of 1** (see [more about them here](#)), and make sure that the *impact* is the same for all features, whether they are very large things, and then very small things, and want to work with all of them.

If on the contrary you would rather have them distributed between 0 and 1, you can use **min max scaling** instead:

$$X_{ij}^U = \frac{x_{ij} - \min_i x_{ij}}{\max_i x_{ij} - \min_i x_{ij}}.$$

Very often in Data Science, and especially using the K-means algorithm, (for instance, one feature might be the age of a person, and another the number of daily cups of coffee). When trying to make calculations on such datasets, it is important to think about the scale of the features. For example, as ages will be greater numbers than the number of daily cups, and you want to **normalise** before using other algorithms (after which, for instance, the weights of the features will be updated, preserving proportions, and allowing easier calculation between the two features).

Examples

Normalisation using one method or another is widely used in Data Science. Here, where min max scaling is used, taken from [here](#):

- "k-nearest neighbors with an Euclidean distance measure if want a distance measure"
- k-means (see k-nearest neighbors)
- logistic regression, SVMs, perceptrons, neural networks etc. if you don't normalise, otherwise some weights will update much faster than others
- linear discriminant analysis, principal component analysis, kernel PCA